

AI EMAIL

TRIAGE SYSTEM

Complete Solution Presentation

SOLUTION OVERVIEW

- Automates full inbox triage
- AI-powered RL environment
- 3 progressive difficulty tasks
- OpenEnv-compliant API
- Docker & Openrouter ready
- Step-by-step reward signals

PROJECT AT A GLANCE

What is the AI Email Triage System?

The **AI Email Triage System** is a real-world AI-powered email management system that integrates with Gmail to process live emails.

The system connects to the user's Gmail account using the Gmail API and retrieves incoming emails in real time. An AI agent then performs the following tasks:

- Classifies each email (spam / important / normal)
- Assigns the correct priority (high / medium / low)
- Generates a professional reply where appropriate

The system uses **Open Router API** to access advanced language models for intelligent classification and response generation. Unlike simulation-based environments, this system operates on real email data, making it practical and applicable in real-world scenarios. It enables automation of inbox management by combining Gmail API for data access and AI models for decision-making.

🎯Goal: Train AI agents that can handle real email triage tasks with human-level accuracy — automatically, consistently, and at scale.

Problem Solution

Approach: Reinforcement Learning Environment

1 Reset & Observe

Environment resets with a queue of emails. Agent receives full email context: subject, body, sender, and valid actions.

2 Take Action

Agent classifies the email, assigns priority, and/or generates a reply using the Action API.

3 Receive Reward

Every action yields an immediate reward signal: +0.40 for correct classify, +0.30 for priority, +0.30 for good reply.

4 Advance & Learn

Environment auto-advances to next email. Grader evaluates full episode score with an optional completion bonus.

TECHNOLOGIES USED

Python 3.9+

Core language for environment, graders & dataset logic

OpenAI API

Language model inference for the baseline agent

Docker

Containerised deployment — runs anywhere consistently

OpenEnv Spec

Standardised reset/step/state API for RL environments

FastAPI

REST API wrapper for Hugging Face Space deployment

Open Router

Deployed via Docker on port 7860 with OpenRouter API for AI processing

RL Framework

Reinforcement learning with dense, step-level rewards

Grader System

Deterministic, weighted accuracy scoring per task

IMPLEMENTATION: THREE TASK LEVELS

Progressive difficulty — from simple classification to full triage with reply generation

TASK 1	EASY
Email Classification	
Goal:	Classify each of all emails as Important, Medium or normal
Actions:	1 action/email: classify_email
Scoring:	Score = Classification Accuracy (0–1)
Limit:	Max 30 steps
Rewards:	+0.40 correct | -0.30 wrong
Challenge:	Distinguishing borderline cases — work newsletters vs. important emails

TASK 2	MEDIUM
Classification + Priority	
Goal:	Classify AND assign priority (high/med/low) to each email
Actions:	2 actions/email: classify + assign_priority
Scoring:	Score = 0.5×classification + 0.5×priority
Limit:	Max 40 steps
Rewards:	+0.30 correct | +0.10 off-by-one | -0.20 wrong
Challenge:	Important emails aren't always high priority. Context matters.

TASK 3	HARD
Full Triage	
Goal:	Classify, prioritise, AND generate a professional reply
Actions:	3 actions/email: classify + priority + reply
Scoring:	Score = 0.35×classify + 0.35×priority + 0.30×reply
Limit:	Max 60 steps
Rewards:	+0.30 good reply | -0.10 bad reply
Challenge:	Reply scored on relevance, tone, and completeness

REWARD SYSTEM & GRADER ARCHITECTURE

Step-Level Reward Table

Action	Reward
Correct classification	+0.40
Wrong classification	−0.30
Correct priority	+0.30
Priority off by one	+0.10
Wrong priority	−0.20
Good reply (≥50 chars)	+0.30
Partial reply (20–50)	+0.10
Bad/empty reply	−0.10
Skip email	−0.20
Task completion bonus	+1.00 × score

Grader Score Formula

Task 1 = Classification Accuracy

Task 2
= 0.5 × Classify
+ 0.5 × Priority

Task 3
= 0.35 × Classify
+ 0.35 × Priority
+ 0.30 × Reply Quality

Reply = 0.45×Relevance + 0.30×Tone + 0.25×Completeness

KEY FEATURES

Dense Reward Signals

Unlike sparse-reward envs, every single action gets an immediate reward — enabling faster learning.

Curriculum Learning

3 tasks of increasing difficulty let agents progressively build triage skill from classification to full reply.

Deterministic Graders

100% reproducible scoring. Same agent always gets same score — no randomness in evaluation.

OpenEnv Compliant

Standard reset()/step()/state() API. Drop-in compatible with any OpenEnv-based training pipeline.

Realistic Dataset

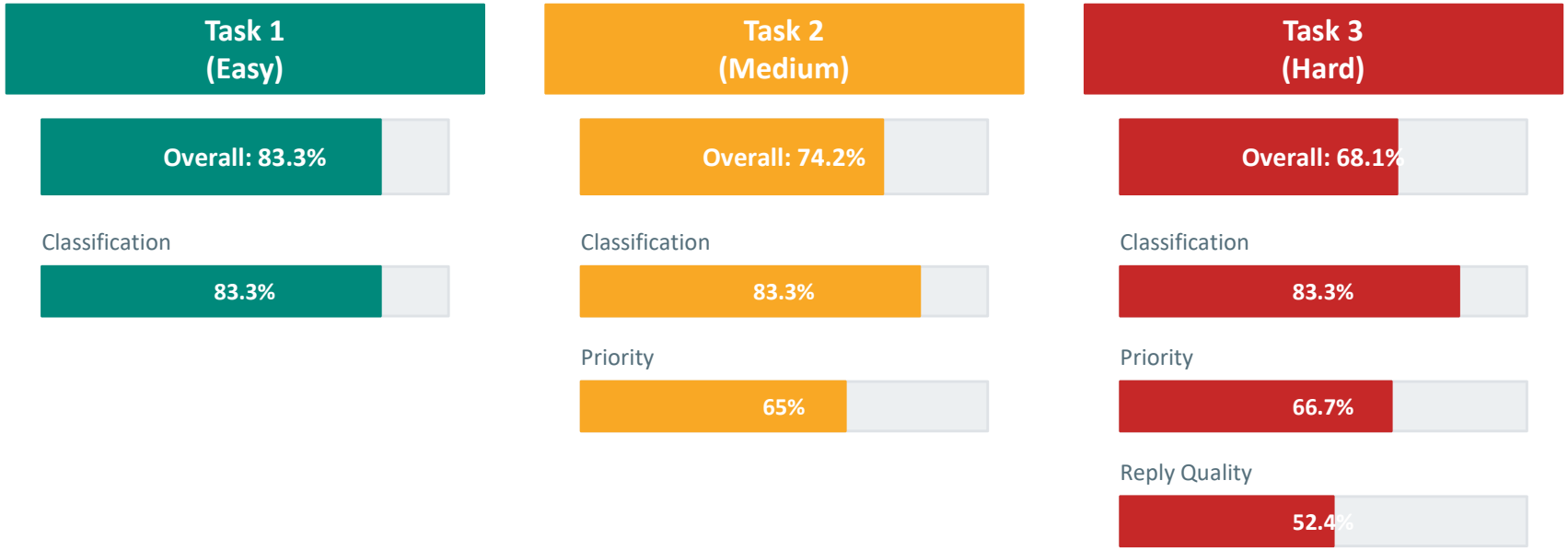
13 curated emails: spam, work-critical, and personal. Designed to test edge cases humans struggle with.

Fully Extensible

Add new emails, tasks, or eval dimensions without restructuring — just follow the schema and register.

BASELINE RESULTS & PERFORMANCE

Evaluated with GPT-4o-mini @ temperature=0.0



Stronger models (GPT-4o, Claude 3.5 Sonnet) are expected to score 0.90+ on Task 1 and 0.75+ on Task 3.

What makes the AI Email Triage System uniquely valuable

Real-World Problem

Email overload is not a toy problem — it affects 4B+ users daily. Our system targets genuine, high-value automation.

Multi-Dimensional Evaluation

Unlike single-metric benchmarks, we evaluate classification accuracy, priority judgment, AND reply quality together.

Built for Agent Training

Dense rewards at every step enable efficient RL training — not just evaluation. Agents can improve continuously.

Plug-and-Play Integration

OpenEnv-compliant API means it slots into any existing training pipeline without custom integration code.

Production-Ready

Docker support, Hugging Face deployment, FastAPI REST wrapper — ready to ship to real users, not just researchers.

Lightweight & Accessible

No GPU needed. Runs on 2 vCPU / 8 GB RAM in under 20 minutes. Accessible to everyone.



THANK YOU

AI Email Triage System — Complete Solution

✓ Real-world problem tackled with Reinforcement Learning

✓ 3 progressive tasks: classify → prioritise → reply

✓ OpenEnv API | Docker | Hugging Face ready

✓ Dense rewards + Deterministic grader scoring

Built with Python · OpenEnv · Docker · Open router